

Asymptotic 1.8.0

A non-duplicative repository audit

Certificate scope, sparse-sector execution,
Wolfram Language comparison, and targeted repairs

Prepared for Vladimir Reshetnikov

Repository `VladimirReshetnikov/Asymptotic`

Pinned revision `921387e5ba1239bfda96e63e64e89bf63d9c41e6`

Review date 9 September 2026

Native runtime Wolfram Language 15.0.0, Linux x86-64

Principal result. An unmodified package can certify a root at $x = 1$ while simultaneously claiming that the entire verification interval satisfies $x < 1/4$. The trigger is an ambient assumption consumed by a domain-proof helper. This is a materially stronger consequence of a previously reported assumption-provenance problem. A separate native reproduction shows that exact flat-series multiplication charges 81 sector pairs for an operation with one active pair.

Two narrow source changes repair the demonstrated paths. A combined native check of ten targeted conditions passed after those changes. Twenty independent mathematical checks and twenty Python-tool/proof-checker tests also passed. These are distinct evidence sets, not a full upstream test-suite result.

The article compares the current implementation with native facilities, maps its findings to the nine existing reviews, provides exact witnesses and proposed fixes, and includes an executable, deliberately restricted domain-proof checker. It does not claim exhaustive formal verification or a fully validated replacement release.

Contents

1	Executive assessment	2
1.1	Recommendation	2
1.2	New findings and their relationship to prior work	2
1.3	What the audit establishes	3
2	Snapshot, method, coverage, and non-duplication	3
2.1	Immutable source and runtime identity	3
2.2	How the existing reviews were used	3
2.3	Coverage and evidence levels	4
3	Implemented capabilities and the Wolfram Language boundary	5
3.1	What the package adds	5
3.2	Feature comparison	5
3.3	Specialized families worth distinguishing	7
3.4	Why native comparison must match contracts	7
4	Native comparison experiments	8
4.1	Ordinary polynomial inversion: exact overlap	8
4.2	Logarithmic forward expansion: native support is real	8
4.3	Irrational-power inversion: a concrete successful specialization	8
4.4	A positive specialized numerical control	9
5	N01: a certificate can verify the wrong source domain	9
5.1	The exact claim being contradicted	9
5.2	Reproducer and native outputs	10
5.3	An elementary proof that the certificate scope is false	10
5.4	Source-level cause: assuming the obligation itself	11
5.5	Why this materially sharpens the existing C05 finding	11
5.6	Minimal repair and native validation	11
5.7	Residual risks and the boundary of the patch	12
5.8	A related retry-policy inefficiency	12
6	N02: a sparse flat product pays for nonexistent pairs	12
6.1	Representation and the specific preflight mismatch	12
6.2	Native public-path witness	13
6.3	The narrow source change	13
6.4	Why exact zero is the indispensable condition	14
6.5	Cost model and limitations of the measurement	14
6.6	A user-visible diagnostic improvement	15
7	Concrete development proposals beyond the two edits	15
7.1	A scoped, replayable domain proof record	15
7.2	Terminal, retryable, and unsupported domain outcomes	16
7.3	Make the certificate boundary independent of notebook state	17
7.4	Budget counters that describe actual sparse execution	17
7.5	A focused development sequence	17
8	Validation results and limits	18
8.1	Final combined native check	18
8.2	Independent mathematics and tooling	18
8.3	Coverage still required before release	19
9	Using the accompanying artifacts	19
9.1	Directory contents	19
9.2	Preview and apply the canonical changes	20
9.3	Run the native focused checks	20
9.4	Run independent checks and rebuild the article	20
10	Conclusion	21
A	Source coverage and prior-review map	21
B	Interpreting negative results and nonclaims	22

1 Executive assessment

1.1 Recommendation

Retain the package’s current direction: it offers useful exact-weight inversion and an inspectable, remainder-aware representation of selected real asymptotic germs. The immediate new priority is narrower than another general stabilization program: *prevent environmental assumptions from discharging a certificate’s own interval obligations*. A successful numerical enclosure of the unrestricted equation is not a certificate for the requested restricted inverse branch. The distinction is violated by the native witness in Section 5.

The second priority is to charge the flat multiplication routine for the pairs it actually visits. Its implementation already omits exact-zero sector pairs, but the preceding budget gate still charges the full rectangular sector grid. The defect is an unnecessary refusal, not a wrong coefficient. A small rearrangement of the gate makes a sparse exact identity calculation succeed without raising the budget or weakening the exact-zero test; see Section 6.

The pre-existing review collection already contains extensive recommendations on native export, precision planning, coefficient algebras, CI, API consistency, and broader transseries development. This report does not repackage that backlog as new work. The added proposals are tied to the two observed paths: explicit quantified domain witnesses, terminal-versus-retryable domain failures, active-pair accounting, and concrete acceptance tests.[2, 3]

1.2 New findings and their relationship to prior work

ID	Priority	Result	Novelty and evidence
N01	Critical to certificate soundness	Ambient assumptions can make InverseCertificate certify an interval outside its retained source domain.	Sharpening of registry C05; native wrong-domain certificate, exact contradiction, and native-tested repair.
N02	Medium	A flat multiplication with one active pair is rejected because 81 dense-grid pairs exceed a budget of 60.	Specific additional insight beyond broad budget item P06; new public-path witness and native-tested repair.

“Critical” here concerns the logical strength of the certificate claim. It is not a security-vulnerability score, a claim of code execution, or evidence that ordinary expansions generally fail. N02 is a performance/availability and diagnostic issue; no mathematical wrong answer is asserted for the rejected product.

The most useful follow-through from N01 is also an observable performance improvement opportunity: an unchanged rational affine domain contradiction is retried at seven enclosure orders in the baseline witness. This is not counted as a third independent bug. It exposes a lost distinction between an obligation that is false and an obligation that may become provable with tighter arithmetic.

1.3 What the audit establishes

The unmodified, pinned standalone was loaded in a native Wolfram kernel. Both findings were reproduced there. The two proposed source edits were then applied to temporary standalone text, loaded normally with `Get`, and checked in the same native runtime. A final combined batch returned `True` for all ten targeted checks. Separate native comparisons established matching ordinary polynomial and logarithmic coefficients and recorded one unevaluated native irrational-power inversion request.

The complete upstream suite was *not* run. Neither a full local checkout nor a newly rebuilt canonical distribution was obtained in the container. The source patcher was tested on explicit anchor fixtures, and the native kernel tested equivalent patched standalone text. These are useful but different validations. The evidence files preserve the distinction, including an initially invalid endpoint test fixture that was corrected before the final successful batch.

2 Snapshot, method, coverage, and non-duplication

2.1 Immutable source and runtime identity

The audit is pinned to `921387e5ba1239bfda96e63e64e89bf63d9c41e6`. The maintained implementation is the modular kernel under `AsymptoticInverse/Kernel/`; the repository-root `AsymptoticInverse.wl` is its generated standalone distribution. The package advertises version 1.8.0 and a Wolfram Language 15.0-or-later requirement. The review uses the pinned source, not an assumption that an earlier report's source still equals current `main`.^[1, 4, 3]

The native runtime identified itself as:

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026)
```

The successful loading route imported the pinned URL as text, exported that text to a temporary file, and used ordinary `Get`. The imported string had 574,893 characters. The SHA-256 of that *exported working copy* was:

```
651a824cac83243f97e6a80ccf50f9c24b915763f32b681932d8f9c269fb21c8
```

After the two changes, the final combined-check working copy had SHA-256:

```
f5254c3abc07f169f1dc48461472001c492d6446265e9055ad5b62156f4484bf
```

These are not claimed to be hashes of raw GitHub response bytes or Git blob identifiers. Text import/export can normalize representation. A Git commit identifies a repository snapshot; a loaded-file hash identifies the particular bytes passed to `Get`. The archive records both kinds of information separately.

Several evaluator calls failed with transport/service errors, while subsequent calls succeeded. Those failures are not evidence of a package timeout, mathematical refusal, or performance limitation. The successful outputs, rather than the failed transports, support the findings.

2.2 How the existing reviews were used

The nine packages under `code-review/` were first mapped through their maintained index and consolidated finding/status registry. Their executive finding inventories and relevant arguments were then inspected, with closer attention to the assumption-provenance, certificate-scope, sparse-operation, and resource-budget discussions. This is a review of their relevant conclusions, not a

claim that every line of every archived script or every appendix was reread.[2, 3, 16, 17, 18, 19, 20, 21, 22, 23, 24]

The registry already distinguishes resolved issues from open work. In particular, it records fixes for the dense native-export allocation issue C01 and the shared pure-remainder fractional-power guard C02. This report does not present either as an unresolved discovery. Its own fixes are in different modules. The ongoing native-export log-tail issue, active nonlinear boundary precision, general assumption provenance, and valuation-sensitive refinement remain prior findings; repeating their original demonstrations would add little.[3]

Prior subject	Treatment here	Additional contribution
C01/C02: native export allocation and fractional remainder powers	Acknowledge the current fixes; do not count again.	None claimed.
C05: ambient assumptions absent from stored provenance	Retain credit to prior reviews, notably R3 and R8.	Construct the object in a clean environment, then corrupt only certificate-time domain verification; obtain an explicitly false certificate scope.
P06: heterogeneous resource budgets	Do not repeat the general resource-policy critique.	Exhibit the exact preflight gate, its already-sparse execution loop, a one-pair public witness, and a narrowly validated change.
D04 and V01: certificate interval UX and release validation	Do not restate their recommendations as new.	Supply concrete new contract tests and distinguish an invalid test center from a product defect.
X06: proof records/checkers	Do not propose a generic proof-record project again.	Implement and test one small replayable affine-domain witness format addressing the observed certificate error.
Other precision, scale, compatibility, and generalization items	Leave in the existing backlog.	Discuss implemented capabilities where needed for a fair comparison, without relabeling old limitations as discoveries.

2.3 Coverage and evidence levels

The source inspection followed the ordinary core, shared series operations, certificate machinery, flat-sector operations, native special-function import, exact special-function identities, Dirichlet adapters, Fourier coefficients, Gamma inverse operations, exponential-core perturbations, and numerical inverse checks. It also used the broader standalone dispatch and the development/review records. Appendix A identifies the depth of the main reads.

Five evidence levels are kept separate. *Native observations* are outputs returned by a Wolfram kernel. *Source deductions* identify an actual control path or invariant. *Independent checks* are exact Python/SymPy calculations or tests of the proposed tooling. *Upstream records* are the repository’s own status/validation claims, not new runs. *Proposals* are design changes whose scope is explicitly stated. No source-only suspicion is promoted to a reproduced runtime bug.

The inspection was risk-directed rather than a line-by-line proof of all 38 kernel modules. It deliberately considered and rejected some plausible suspicions. For example, the exponential-core

inverse does retain the "Power" field required by numerical checking; a native positive control succeeded. A truncated source excerpt was not treated as evidence that the field was absent. This matters in a large association-based implementation where constructors often attach important metadata near their end.

3 Implemented capabilities and the Wolfram Language boundary

3.1 What the package adds

The central object is not just an expression containing a few small terms. An ordinary `GeneralizedSeries` represents a selected real germ using a positive coordinate $u \rightarrow 0^+$, an ordered set of exact weights, complete logarithmic coefficient blocks, and a remainder. At the jet level the representation is

$$J(u) = \sum_{\beta \in B} u^\beta P_\beta(\log u) + \mathcal{O}(u^\rho(1 + |\log u|)^d). \quad (1)$$

The powers need not share one rational denominator. Exact weight comparison and merging precede truncation, so equal-weight contributions are treated together. A more general result also retains an exact offset, an exact prefactor/carrier, source and target conditions, and data for refinement or specialized operations.[4, 5]

At a finite endpoint the coordinate is a signed source distance; at positive or negative infinity it is a signed reciprocal chosen positive on the real approach. The ordinary inverse engine normalizes a model

$$f(u) = y_0 + au^p \left(1 + \sum_j u^{\delta_j} Q_j(\log u) \right), \quad \delta_j > 0, \quad (2)$$

and works in a selected real target uniformizer. Lagrange, grouped Lagrange, and Newton-related computation are available. The incremental implementation retains state by value, advances complete weight layers, and checks normalized residuals. These mechanisms already exist; cached coefficients, exact-weight grouping, and Newton residual checking are not proposed here as missing features.[4, 6]

This specialization is especially useful when the result must subsequently be multiplied, differentiated under a valid regularity contract, refined, or compared with another scale without silently dropping the error information. It does not make every advertised family part of one unrestricted transseries field.

3.2 Feature comparison

The following table separates native capabilities from the package's additional representation and algorithms. It is not a claim that every listed native function accepts every parameter choice, nor that every recognized package head is supported at every endpoint.

Area	Native Wolfram Language	Pinned package and practical distinction
Ordinary series	<code>Series</code> , <code>SeriesData</code> , arithmetic, composition, integration and differentiation.	Adds explicit real-germ metadata and the power/log remainder pair; ordinary examples often overlap exactly.

Area	Native Wolfram Language	Pinned package and practical distinction
Reversion	<code>InverseSeries</code> reverts suitable native series, including ramified examples.	Adds specialized exact-real-weight inversion, complete log blocks, method/state inspection, and selected inverse-scale families.
Implicit asymptotics	<code>AsymptoticSolve</code> handles implicit equations and systems, including real-domain requests.	Focuses on selected one-variable real inverse germs and provides a reusable result object. One failed native input form is not a universal capability theorem.
Logarithmic coefficients	Native series explicitly allow logarithms among coefficients.	Records logarithmic error degree separately and manipulates complete blocks. Native logarithms are not a missing feature.
Irrational weights	General native expressions and asymptotic inference can contain nonrational powers; one <code>SeriesData</code> lattice is not an arbitrary ordered real support.	Has an explicit ordered sparse exact-weight calculus; the tested irrational inverse succeeds directly.
Exact Lambert cores	<code>ProductLog</code> represents Lambert branches.	Retains selected exact cores inside subsequent inverse corrections instead of always expanding them away.
Special functions	<code>Series</code> and <code>Asymptotic</code> provide broad native coefficient generation.	Adapters add real-domain checks, carrier separation, exact identity normalization, and explicit error transport; much coefficient generation is delegated to the native kernel.
Gamma/Barnes inverses	Native special functions and Lambert expressions provide ingredients.	Dedicated positive-argument inverse constructions use finite Poincaré models and retained reciprocal-log coefficient blocks.
Flat sectors	Native expressions can retain exponentials and rapidly varying factors.	Provides a finite sector representation and dedicated multiplication, observables, inner truncation, and qualified differentiation.
Log-phase Fourier coefficients	Native trigonometric/exponential algebra is available.	Separates exact frequencies from source weights, merges resonances, and applies hard frequency budgets without silently dropping modes.
Validation	Native symbolic relations and numerical solvers offer different evidence.	Separates model residuals, numerical comparisons, and exact rational interval certificates for admitted explicit functions; N01 shows why the last layer needs scope hardening.
Broader analysis	Native asymptotic functions address integrals, transforms, differential equations, and systems.	This repository is not a general replacement for those facilities or for complex/uniform/multivariate asymptotics.

The native columns are grounded in the current documentation for `Series`, `SeriesData`, `InverseSeries`, `ComposeSeries`, `Asymptotic`, `AsymptoticSolve`, and `ProductLog`. The package columns follow the cited modules rather than an old roadmap.[25, 26, 27, 28, 29, 30, 31, 4, 5, 8, 12, 13, 9]

3.3 Specialized families worth distinguishing

Native-series adapters. The special-function importer is an integration layer, not an independent implementation of all Bessel, Airy, error-integral, hypergeometric, and elliptic asymptotics. It reads structured native output, propagates error terms, separates exact carriers, and checks real-domain conditions. When a native expression has no remainder, the inspected path does not simply declare it exact without a source-equality check. Exact special-function identities are applied before finite asymptotic truncation so that a terminating dominant expression need not erase a second exponential contribution.[9, 10]

Dirichlet and moment expansions. The Dirichlet adapter is a different mechanism. For real $s > 1$, truncating the defining zeta series before m leaves

$$m^{-s} \leq \sum_{n=m}^{\infty} n^{-s} \leq m^{-s} \left(1 + \frac{m}{s-1}\right). \quad (3)$$

The upper bound follows by bounding the terms after m with an integral. With $u = e^{-s}$ the individual terms become $u^{\log n}$, an instructive exact-weight scale. The Lerch path instead expands in a large argument with fixed $|z| < 1$, using finite moments of a geometric series and a controlled Taylor tail. These endpoint-specific constructions should not be conflated with a generic native import.[11]

Gamma and Barnes inverses. At large normalized logarithmic target L , the retained cores are

$$X_{\Gamma} = \frac{L}{W(L/e)}, \quad X_G = \sqrt{\frac{4L}{W(4L/e^3)}}. \quad (4)$$

The first solves $X \log X - X = L$. The second corresponds to the quadratic-logarithmic leading Barnes balance. Corrections use complete polynomials in reciprocal core logarithms at successive inverse-core powers. The implementation describes finite Poincaré constructions, not convergent inversion of an infinite Stirling series or a pointwise interval certificate.[13]

Flat and Fourier scales. A flat object has a finite set of exponential sectors and separate inner power/log uncertainty. A Fourier coefficient has finite modes $P_{\omega}(\log u)e^{i\omega \log u}$. Differentiation of a Fourier coefficient uses $P'_{\omega} + i\omega P_{\omega}$, while reality is checked by conjugate pairing. Neither an oscillatory leading term nor an arbitrary unknown remainder automatically provides a nonzero real denominator. The package's guarded finite algebras are valuable precisely because they avoid pretending that those operations are universally available.[8, 12]

3.4 Why native comparison must match contracts

The correct comparison is not simply equal input integers. An exclusive package cutoff of 6 and a native polynomial series through degree 5 can describe the same retained polynomial. A `SeriesTermGoal` may count blocks rather than a polynomial degree. A retained exact core may encode infinitely many elementary logarithmic terms. Two outputs can therefore be mathematically comparable without having the same printed term count.

Likewise, equality of `Normal` expressions checks only finite coefficients. It does not establish equality of domains, remainder classes, regularity assumptions, or numerical validity thresholds. The experiments below deliberately report both finite expressions and the package's remainder.

They do not infer a timing advantage, because no controlled head-to-head timing experiment was performed.

4 Native comparison experiments

4.1 Ordinary polynomial inversion: exact overlap

The comparison uses the same forward function and matches the truncation boundary:

```
p = AsymptoticInverse[x + x^2, {x, 0}, {y, 6}];
n = InverseSeries[Series[x + x^2, {x, 0, 5}], y];
{Normal[p], Normal[n], Simplify[Normal[p] - Normal[n]]}
```

Both returned

$$y - y^2 + 2y^3 - 5y^4 + 14y^5, \quad (5)$$

and the difference simplified to zero. The package remainder was `PowerLogRemainder[y, 6, 0]`. Independently, expanding $(\sqrt{1+4y}-1)/2$ produces the same coefficients, and substituting the polynomial into $x + x^2 - y$ leaves no term below degree 6.

This is a positive native baseline, not evidence of a package advantage on ordinary reversion. For this class, native `InverseSeries` is already a natural tool. The package’s additional value lies in the retained metadata and the wider specialized workflows, not in the mere existence of these five coefficients.^[27]

4.2 Logarithmic forward expansion: native support is real

The second comparison was

```
p = AsymptoticExpansion[x^x, {x, 0, 3}];
n = Series[x^x, {x, 0, 2}];
```

Both finite expressions were

$$1 + x \log x + \frac{x^2 (\log x)^2}{2}, \quad (6)$$

with a zero simplified difference. The package returned

```
PowerLogRemainder[x, 3, 3]
```

which accords with $x^x = \exp(x \log x)$ on $x > 0$. Since $x \log x \rightarrow 0$, the next exponential Taylor term gives an error of order $x^3 |\log x|^3$.

The conclusion is narrower and more useful than a slogan about “logarithmic series support”: both systems compute these coefficients, while the package explicitly retains a log-degree component in its error descriptor. Native documentation itself uses x^x to illustrate logarithms among series coefficients.^[26]

4.3 Irrational-power inversion: a concrete successful specialization

The third package request was

```
AsymptoticInverse[x + x^Sqrt[2], {x, 0}, {y, 2}]
```

It returned

$$y - y^{\sqrt{2}} + \sqrt{2} y^{2\sqrt{2}-1} + \mathcal{O}(y^{3\sqrt{2}-2}), \quad y \downarrow 0. \quad (7)$$

The stored remainder exponent was exactly $3\sqrt{2} - 2$, not a decimal approximation. It exceeds the requested exclusive cutoff 2; the engine exposed the next actual support weight.

For a direct native comparison, the following bounded request was evaluated:

```
TimeConstrained[
  AsymptoticSolve[x + x^Sqrt[2] == y, x -> 0, y -> 0,
    Reals, SeriesTermGoal -> 3],
  8, "BoundedTimeout"]
```

It returned the `AsymptoticSolve` expression unevaluated; it did not return the timeout marker. This establishes the behavior of *this input form on the tested 15.0.0 kernel*. It does not establish that every reformulation, manual normalization, or future/native version must fail. In particular, native `AsymptoticSolve` documents automatically inferred nonclassical scales and substantially broader equation/system inputs.[\[30\]](#)

There is an independent exact derivation of the package coefficients. Put $\alpha = \sqrt{2}$, $t = y^{\alpha-1}$ and $x = yh(t)$. Then

$$h + th^\alpha = 1, \quad h(0) = 1. \quad (8)$$

The analytic implicit-function theorem near $(h, t) = (1, 0)$ applies because the derivative with respect to h is 1 there. Lagrange inversion, or direct coefficient comparison, gives

$$h(t) = 1 - t + \alpha t^2 - \frac{\alpha(3\alpha - 1)}{2} t^3 + \mathcal{O}(t^4). \quad (9)$$

Multiplication by y gives (7), including a nonzero first omitted coefficient. The independent SymPy check composes the cubic h back into its equation and verifies cancellation through t^3 . Thus the native success is supported by a mathematical oracle that does not reuse the package's coefficient routine.

4.4 A positive specialized numerical control

An additional native control constructed

```
s = AsymptoticExponentialCoreInverse[
  Exp[x], x, {x, Infinity}, {y, 2}];
InverseNumericalCheck[s, Exp[10] + 10, WorkingPrecision -> 30]
```

The stored object contained the "Power" field, and the numerical comparison succeeded with a reference root approximately 10. The approximation error was about 1.8074×10^{-11} , the reported remainder scale about 9.3462×10^{-11} , and their ratio about 0.19338. These are observed numerical values, not an interval proof or a general bound constant. They are retained in the ledger as a useful control and as a record of a suspected metadata issue that was rejected.[\[14, 15\]](#)

5 N01: a certificate can verify the wrong source domain

5.1 The exact claim being contradicted

`InverseCertificate` is stronger than an asymptotic expansion or a numerical comparison. For admitted explicit functions it returns a unique-root enclosure on a verification interval, with exact rational interval arithmetic and an asserted source-domain check. The helper's own comment says that a Boolean domain must hold on the entire closed interval, including strict inequality behavior at endpoints.[\[7\]](#)

The defect is not that a source branch is merely local or that an asymptotic approximation is poor far from its endpoint. The API explicitly checks an interval for a fixed target and reports a verification result. In the witness, it reports both

```
"CertifiedSourceDomain" -> x < 1/4
"SourceDomainVerified" -> True
```

while its root enclosure is the singleton $\{1\}$. Those statements cannot all describe a valid certificate for the restricted positive inverse germ.

5.2 Reproducer and native outputs

Construct the object *outside* the polluted assumption scope:

```
s = AsymptoticInverse[
  ConditionalExpression[x + x^2, x < 1/4],
  {x, 0}, {y, 3}];

clean = InverseCertificate[s, 2,
  "Interval" -> {9/10, 11/10}, "Center" -> 1];

polluted = Block[{$Assumptions = x < 1/4},
  InverseCertificate[s, 2,
    "Interval" -> {9/10, 11/10}, "Center" -> 1]];
```

The clean call returned `Failure["OutsideBranch",...]` with `"Certified" -> False`, the retained condition $x < 1/4$, and the requested interval. The polluted call instead returned the following selected fields:

```
<|"Certified" -> True,
  "RootEnclosure" -> {1, 1},
  "Center" -> 1,
  "CertifiedErrorBound" -> 0,
  "ResidualEnclosure" -> {0, 0},
  "VerificationInterval" -> {9/10, 11/10},
  "CertifiedFunction" -> x + x^2,
  "CertifiedTarget" -> 2,
  "CertifiedSourceDomain" -> x < 1/4,
  "SourceDomainVerified" -> True,
  "Route" -> "OriginalFunction"|>
```

The complete output also described its scope as a root on the selected source side within every retained source-domain condition. The field subset above is transcribed, not a fabricated expected result. The archive includes the reproducer and selected native observations.

5.3 An elementary proof that the certificate scope is false

Proposition 5.1. *The polluted result is not a certificate for the selected restricted inverse branch of $f(x) = x + x^2$ at 0 from above.*

Proof. The selected positive source branch together with the retained condition is $0 < x < 1/4$. On this interval $f'(x) = 1 + 2x > 0$, and

$$0 < f(x) < \frac{1}{4} + \frac{1}{16} = \frac{5}{16} < 2.$$

Consequently $f(x) = 2$ has no root on that branch. The claimed root $x = 1$ satisfies the unrestricted equation but violates $x < 1/4$. Moreover, every point of $[9/10, 11/10]$ exceeds $1/4$,

so the claimed whole-interval domain verification is false, not merely too weak to determine membership of the root. $\square$

This proof needs no numerical conditioning argument. The exact residual at $x = 1$ is zero, and the derivative is positive on the verification interval. Those facts correctly establish an unrestricted root, but cannot establish membership in a domain that excludes the entire interval. Increasing precision is irrelevant to the contradiction.

5.4 Source-level cause: assuming the obligation itself

The decisive line is in `certPositiveCondition`:

```
c = TimeConstrained[
  Quiet[Refine[condition, Element[x, Reals]]],
  1, condition];
```

It then accepts `c === True` before computing interval ranges. The positional realness assumption is not an isolation boundary: Wolfram's documented default also uses `$Assumptions`. A native check returned `True` for the predicate $x < 1/4$ when the same predicate was placed in the ambient assumption variable.[\[7, 32\]](#)

The logical distinction is

$$\forall x \in [a, b]: \phi(x), \quad (10)$$

$$\phi(x) \implies \phi(x). \quad (11)$$

The first is the certificate obligation. The second is the tautology that the polluted refinement can discharge. Passing `Element[x, Reals]` as an additional positional assumption does not turn (11) into (10). The code's later affine interval calculation is never reached once the condition becomes `True`.

This is why simply recording more assumptions in the returned certificate would not be an adequate fix. A condition on the interval variable must be checked over the interval. It cannot be treated as an unrestricted external premise while simultaneously being reported as verified there. Parameter hypotheses and quantified-variable obligations require different roles.

5.5 Why this materially sharpens the existing C05 finding

Earlier reviews demonstrate that ambient assumptions can influence construction without being fully represented in stored provenance. N01 isolates a different moment in the workflow: the series object is constructed cleanly, and only the certificate call sees the additional assumption. Thus recording the constructor's environment would not repair this witness.[\[18, 23, 3\]](#)

The impact is also stronger. An incompletely qualified coefficient formula is a provenance defect. Here a proof-producing interface returns an unconditional `Certified -> True` result with an explicit, false domain-verification field. The rational enclosure arithmetic may remain correct; the total certificate assertion does not. This qualifies earlier favorable assessments of the certificate machinery without misrepresenting them as prior full formal-verification claims.

5.6 Minimal repair and native validation

The supplied patch makes the local refinement's option explicit:

```
Refine[condition, Element[x, Reals], Assumptions -> True]
```

The second argument retains the intended realness hypothesis; the explicit option prevents this call from importing the ambient assumption variable. That behavior is documented and was confirmed natively. The rest of the interval predicate evaluation is unchanged.[32]

After applying this change to a temporary standalone, the polluted witness returned `OutsideBranch`. A positive control with target $11/100$, interval $[9/100, 11/100]$, and center $1/10$ still certified successfully, and its rational root enclosure contained the exact root $1/10$. A strict-endpoint control with target $5/16$, interval $[1/5, 1/4]$, and interior center $9/40$ was refused as `OutsideBranch`.

The distinction between *center* and *domain endpoint* matters in the test harness. An initial fixture chose center $1/4$, at the upper endpoint, and correctly received `InvalidCenter` before reaching the intended domain check. The fixture was changed to $9/40$; it then tested the desired strict-domain condition. The final combined batch used the corrected fixture. No invalid-input rejection is counted as an additional repository bug.

5.7 Residual risks and the boundary of the patch

This one-line repair closes the demonstrated `Refine` path. It is not a proof that every constructor, simplifier, user definition, option mutation, or future certificate helper is immune to environmental effects. It does not resolve all of registry C05. The broader proposal in Section 7.1 is to make proof context explicit and local, while keeping a narrow, reviewable mathematical checking boundary.

In particular, a stronger API should distinguish a fixed parameter premise from a condition on the source variable being quantified. It should not accept arbitrary source-variable assumptions as substitutes for closed-interval verification. Any future cache of proof results must be keyed by the actual predicate, quantified variable, interval, and permitted parameter context; caching only a simplified Boolean would recreate the same problem in another form.

5.8 A related retry-policy inefficiency

The clean baseline failure contained seven attempts at enclosure orders

$$60, 120, 240, 480, 960, 1920, 2000.$$

Every entry had the same center 1 and the same `OutsideBranch` outcome on the unchanged interval. The relevant affine range is exactly

$$\{x - 1/4 : x \in [9/10, 11/10]\} = [13/20, 17/20]. \quad (12)$$

No rounding refinement can make its upper endpoint negative. The current source already has an exact affine range path; the lost information is that this particular obligation is *disproved*, not merely unresolved.[7]

The proposed change is not “disable certificate refinement.” Refinement remains appropriate when an enclosing range straddles a threshold only because a nonlinear or transcendental bound is too coarse. Instead, preserve a reason category and retry only conditions whose reason permits improvement. This avoids repeated proof work and makes the failure message actionable without weakening any acceptance criterion.

6 N02: a sparse flat product pays for nonexistent pairs

6.1 Representation and the specific preflight mismatch

The flat calculus keeps sectors indexed by an exponential degree, with power-log jets inside each sector. A product combines the sector jets by convolution, while propagating inner uncertainty

and a complete omitted-sector tail separately. It is deliberately not ordinary multiplication of finite expressions stripped of their error terms.^[8]

The implementation already identifies exact-zero sectors and iterates only over active indices. However, before doing so it checks

```
If[(na + 1) (nb + 1) > limit,
  fail["ResourceLimit",
    "The complete flat-sector convolution exceeds MaxTerms pair products."];
```

Only later does it compute

```
aIndices = Select[Range[na + 1], ! flatOpsExactZeroQ[aj[[#]]] &];
bIndices = Select[Range[nb + 1], ! flatOpsExactZeroQ[bj[[#]]] &];
```

and use those lists as the iteration domains. Thus the preflight count and the executed pair loop describe different workloads.

A worst-case reservation policy can be conservative without producing a false formula. Nevertheless, here the exact workload is already cheaply available, the sparse loop is already implemented, and the reservation rejects a trivial valid operation. This is a concrete performance/availability issue, not a criticism that a hard resource limit exists.

6.2 Native public-path witness

```
s = AsymptoticFlatInverse[x + Exp[-1/x], {x, 0}, {y, 8}];
c = FlatSeriesObservable[s, 1, z];

{Normal[c], c["Remainder"],
 c["FlatRepresentation"]["SectorDepth"],
 LeafCount[c["FlatRepresentation"]["SectorJets"]]}

FlatSeriesMultiply[c, 1, "MaxTerms" -> 60]
```

The inspected values were

```
{1, 0, 8, 49}
```

followed by the dense-pair `ResourceLimit` failure. In other words, the input is an exact constant, its remainder is zero, and its retained representation passes the 60-leaf budget because its leaf count is 49. The multiplication nevertheless charges

$$(8 + 1)^2 = 81 > 60$$

sector pairs. Only the zero-degree sector of either factor is nonzero, so the actual number of active pairs is

$$1 \times 1 = 1.$$

This witness deliberately uses a budget of 60, not 45. A 45-budget test legitimately fails earlier because the existing representation contains 49 leaves; that would not demonstrate the pair-count mismatch.

6.3 The narrow source change

The fix computes the active lists before the pair-budget check and replaces the charged count by their product:

```

aIndices = Select[Range[na + 1], ! flatOpsExactZeroQ[aj[[#]]] &];
bIndices = Select[Range[nb + 1], ! flatOpsExactZeroQ[bj[[#]]] &];
If[Length[aIndices] Length[bIndices] > limit,
  fail["ResourceLimit",
    "The active flat-sector convolution exceeds MaxTerms pair products."]];

```

The convolution loop, per-jet multiplication checks, tail propagation, and result-budget validation remain unchanged. No mode, coefficient, or unknown error is silently truncated to fit the budget. The native patched witness returned `GeneralizedSeries`, finite expression 1, and remainder 0 with the same budget of 60.

This is intentionally not a new universal resource-budget framework. Storage, expression growth, proof work, and pair counts still have different costs. Existing broader recommendations on that topic remain in P06. The contribution here is a local count that can be made exact with negligible extra planning work.

6.4 Why exact zero is the indispensable condition

The helper is

```
flatOpsExactZeroQ[j_] := j[[1]] === {} && j[[2]] === Infinity;
```

Both clauses matter. An empty retained support with finite precision represents an unknown remainder, not an exact zero. The native checks returned

```

flatOpsExactZeroQ[{}, Infinity, 0] (* True *)
flatOpsExactZeroQ[{}, 3, 0]       (* False *)

```

The patch leaves this predicate unchanged. Replacing it by an empty-support test would make the optimization unsound.

Proposition 6.1. *For finite sector lists, removing pairs containing a sector whose entire coefficient jet is exactly zero leaves the coefficient convolution and every propagated uncertainty contribution unchanged. The same statement is false for a sector with empty finite support but nonzero unknown remainder.*

Proof. An exact zero coefficient is the zero element of the coefficient algebra and has no uncertainty. Its product with every other coefficient is identically zero. Omitting those pairs therefore changes neither the retained convolution nor its error terms. By contrast, a coefficient represented only by $\mathcal{O}(u^3)$ may be the nonzero function u^3 . Multiplying it by an exact unit still gives u^3 ; discarding that pair would erase a legitimate uncertainty contribution in its own exponential sector. $\square$

The proof justifies the same active lists the source already uses. N02 does not require inventing a new multiplication formula; it requires using the existing safe sparsity criterion in the preflight decision.

6.5 Cost model and limitations of the measurement

Let $n_a + 1$ and $n_b + 1$ be the stored sector counts and let a, b be their active counts. The present gate charges $(n_a + 1)(n_b + 1)$, while the outer loop visits ab pairs. Computing the active lists costs $\mathcal{O}(n_a + n_b)$. For constant-only sector data with a common retained depth N , the pair counts are:

Depth N	Dense gate count	Active loop count
8	81	1
32	1,089	1
128	16,641	1

Only the depth-8 row is a native public-path observation here. The other rows are exact arithmetic illustrations of the count formula, not constructed or timed Wolfram objects.

The change does not make the entire operation constant-time. The stored sector lists, temporary convolution vector, tail calculations, coefficient normalization, and output construction still have costs. In particular, the routine still allocates a vector of length $n_a + n_b + 1$. The unchanged representation and result budgets remain relevant. A claim of an 81-fold speedup would be unjustified: the observation is a change from refusal to success, and the mathematical count concerns only outer pair visits.

6.6 A user-visible diagnostic improvement

The old message says the complete convolution exceeds its budget even though the implementation would never visit most of those pairs. The new message names active pairs, matching the gate’s meaning. A further, small extension would attach "StoredSectorCounts", "ActiveSectorCounts", "RequiredPairs", and the applicable limit to failures. That would let a user distinguish a genuinely dense product from a large padded representation without opening private fields.

Such fields should report an algorithmic count, not an estimated memory footprint or a promised runtime. They should also be computed before expensive coefficient products. The proposal follows directly from this reproduction and is more actionable than another undifferentiated request for “better performance diagnostics.”

7 Concrete development proposals beyond the two edits

7.1 A scoped, replayable domain proof record

The certificate bug is a quantifier/context error. The smallest useful proof-record experiment should therefore record the quantified domain obligation, not attempt to formalize the entire asymptotic engine. For a rational affine predicate $ax + b \leq 0$ on a closed rational interval, an independent checker can recompute the exact attained range from its endpoints and decide the predicate universally.

The package already computes such affine ranges internally. The new proposal is to *retain their evidence and classification* rather than discard them into a Boolean. The accompanying `code/affine_domain_proof.py` implements the following restricted format:

```
{
  "schema": "AffineClosedIntervalProof/1",
  "quantifier": "ForAll",
  "condition": {"a": "1", "b": "-1/4", "relation": "<"},
  "interval": ["9/10", "11/10"],
  "range": ["13/20", "17/20"],
  "status": "Disproved",
  "counterexample": "9/10",
  "assumptions": [],
  "arithmetic": "Exact rational endpoints; affine range is attained"
}
```

No ambient assumption input exists. Numeric fields are exact rational strings or integers; binary floating-point values are rejected. The verifier recomputes the entire expected record from the predicate and interval. Mutating the result status, range, or premise list therefore invalidates the record.

For the six supported relations, the universal tests on an attained range $[L, U]$ are:

Relation	Exact universal criterion
< 0	$U < 0$
≤ 0	$U \leq 0$
> 0	$L > 0$
≥ 0	$L \geq 0$
$= 0$	$L = U = 0$
$\neq 0$	$U < 0$ or $L > 0$

If an affine non-equality crosses zero, a counterexample can be the interior root $-b/a$ rather than an endpoint. The prototype handles that case explicitly. It also handles negative slopes, weak versus strict endpoint contact, and the identically zero affine function.

This prototype is *not* a verifier for a complete inverse certificate. It does not establish existence, uniqueness, special-function bounds, arbitrary nonlinear predicates, or a connection between an arbitrary external certificate and its original function. Its purpose is to make one observed logical obligation independently replayable. Extending it should preserve that small scope rather than turn a sample checker into an unqualified proof-system claim.

7.2 Terminal, retryable, and unsupported domain outcomes

A Boolean domain helper conflates three useful outcomes. A predicate may be proved on the interval; it may be refuted by an exact witness; or its available enclosure may be insufficient. Unsupported syntax is a fourth operationally distinct case. The certificate loop should receive enough information to choose an appropriate next action.

For the observed rational affine counterexample, the outcome is **Disproved** with a fixed witness. Raising enclosure order should not retry it. For a nonlinear expression whose interval enclosure crosses a threshold although the true range may not, **Unresolved** can carry the dependency on arithmetic precision or interval subdivision. For an unsupported function head, the next action is a different enclosure method or an explicit refusal, not seven repetitions of the same unsupported evaluation.

One possible internal record is

```
<|"Status" -> "Disproved",
  "Obligation" -> heldPredicate,
  "Interval" -> {lo, hi},
  "Scope" -> "EntireClosedInterval",
  "Witness" -> exactCounterexample,
  "RetryActions" -> {}|>
```

A proof result and a failure reason should be bound to the same interval. If an adaptive strategy changes the interval, it must recompute that obligation. This is a precise acceptance rule: *no repeated enclosure-order attempt for an unchanged interval with an exact domain counterexample*. It does not require changing the successful rational root-enclosure algorithm.

7.3 Make the certificate boundary independent of notebook state

The immediate **Refine** option fix is preferable to globally changing the user's environment. A broader hardening pass should localize only the proof subsystem and pass explicit permitted hypotheses to every proof-producing symbolic call. It must not mutate **\$Assumptions** outside that dynamic scope or assume that all user-defined functions are trustworthy proof procedures.

A useful metamorphic test begins with one cleanly constructed object and evaluates the same interval certificate under several irrelevant or hostile ambient assumptions. Its accepted root/domain claim should remain invariant unless the API explicitly captures and reports an allowed parameter context. The N01 regression specifically uses the conclusion itself as the hostile assumption. That case is more discriminating than testing only an unrelated parameter hypothesis.

This recommendation is a certificate-specific elaboration of existing assumption-provenance work, not a replacement for it. Constructor-time assumptions, operation-time branch proofs, and certificate-time universal checks remain different scopes. The current counterexample shows that solving one of them does not automatically solve the others.

7.4 Budget counters that describe actual sparse execution

For the flat path, the next useful diagnostic is an exact counter at the same abstraction level as the gate: active outer sector pairs. The count is known before the loop and can be returned in statistics or a failure. Per-jet work remains governed by the inner algebra's controls. Separating those counters avoids pretending that one sector pair necessarily costs one scalar multiplication.

A regression can then assert both the mathematical result and the intended charge. Constant-by-constant multiplication at depth 8 should produce the exact unit under a 60 budget and report one outer pair. A pair involving a pure unknown inner remainder must remain active. A truly dense case whose active-pair count exceeds the budget must still fail before the convolution products are formed.

The supplied patch establishes the first two conditions through native checks and unchanged source predicates. A full dense/tail stress matrix and instrumentation of actual inner work remain follow-up validation tasks, not completed results. The current code changes only the preflight count, so this follow-up is a contained extension rather than a rewrite of the flat calculus.

7.5 A focused development sequence

Milestone A: close the observed paths. Review the two canonical diffs, regenerate the standalone from canonical modules, and run the supplied ten checks plus the upstream suites covering certificates and flat operations. Acceptance requires rejecting N01 under polluted assumptions, preserving a valid interior certificate, respecting strict endpoint conditions, and accepting the sparse identity without discarding unknown inner errors.

Milestone B: preserve failure evidence. Extend the existing affine range path to return a witnessed outcome. Stop increasing enclosure order for an unchanged, exactly disproved domain. Acceptance includes a one-attempt failure for the N01 interval and continued refinement on a genuinely precision-limited enclosure fixture.

Milestone C: expose exact sparse charges. Add active-count diagnostics and exercise sparse, dense, exact-zero, and remainder-only sector patterns. Acceptance includes preallocation refusal on truly excessive active pair counts and unchanged coefficient/tail results on a representative complete flat-operation matrix.

These milestones complement, rather than repeat, the existing roadmap. They intentionally do not add another list of complex sectors, multivariate inversion, integration, transseries closure, or generic CI recommendations already recorded in the review collection.

8 Validation results and limits

8.1 Final combined native check

The final combined native batch used both temporary standalone edits and returned ten successes:

Check	Condition exercised	Observed
C01	Clean invalid-domain certificate is rejected.	True
C02	The same invalid domain is rejected under the hostile ambient assumption.	True
C03	A valid interior-domain certificate still succeeds.	True
C04	Its rational enclosure contains the exact root $1/10$.	True
C05	A strict domain boundary at the interval endpoint is rejected with a valid interior center.	True
F01	The depth-8 exact constant product succeeds with budget 60 and has remainder zero.	True
F02	An exact zero jet is recognized as zero.	True
F03	Empty support with finite unknown precision is not recognized as exact zero.	True
B01	Polynomial inverse coefficients agree with native InverseSeries .	True
B02	Logarithmic forward coefficients agree with native Series .	True

The names are test identifiers, not additional independent findings. The ten-condition batch is not the package's full **TestReport** suite. In particular, F02/F03 directly inspect the existing private zero predicate; they do not by themselves exhaustively test all uncertain-tail convolutions.

8.2 Independent mathematics and tooling

The independent mathematical program completed 20 checks using exact rational arithmetic and SymPy. They include the invalid certificate's domain contradiction, the valid control, the strict endpoint, exact work counts, ordinary inverse composition, and the irrational inverse's normalized implicit equation.

A separate Python test run completed 20 tests: eight source-anchor transformation tests and twelve restricted affine-proof/checker tests. The latter include tampered claims, tampered ranges, added assumptions, interior non-equality witnesses, and rejection of inexact numeric inputs. These counts must not be added to the ten native checks as though all 50 were upstream Wolfram tests.

The patch-tool tests operate on explicit small fixtures. They establish unique-anchor enforcement, fail-closed behavior on missing/duplicated anchors, rejection of reapplication, placement of the active-pair gate, and preservation of the exact-zero predicate. They do not establish that every release/build environment will accept the scripts or that the complete repository has been rebuilt.

8.3 Coverage still required before release

The demonstrated fixes are small, but the successful check set is not enough for a release claim. Certificate validation should include nested Boolean source conditions, non-affine enclosures, both source sides, transformed coordinates, exact and inexact seed selection, and domain predicates involving permitted parameters. The question is whether the isolation change remains local and whether every accepted scope is the one actually checked.

Flat validation should include nonconstant zero sectors where admitted, unequal sector depths, inner truncation, products of uncertain sectors, sector-tail propagation, observables, and derivative contracts. These are not alleged new bugs; they are neighboring behaviors that the narrow spot checks do not cover. A canonical rebuild must also be compared with the tested standalone modifications.

The supplied Git patcher refuses an unpinned or dirty target checkout, but a clean pin and successful edit are not substitutes for those native tests. The safe deployment posture is to treat the artifacts as a reproducible review and a focused patch proposal, not a packaged corrected upstream release.

9 Using the accompanying artifacts

9.1 Directory contents

Path	Purpose
article/asymptotic_delta_audit.tex and .pdf	Self-contained article source and rendered report.
code/patch_spec.json	The exact two source transformations and snapshot pin.
code/apply_fixes.py	Preview or apply canonical-source edits to a clean pinned Git checkout.
code/native_probe.wls	Load a supplied standalone, optionally patch a temporary copy, run the ten checks, and export evidence.
code/regression_checks.wl and tests/DeltaAudit.wlt	Named desired-contract checks and a Wolfram testing wrapper.
code/comparison.wl	Polynomial, logarithmic, and bounded irrational-inverse comparisons.
code/reproduce_certificate.wl	The unmodified wrong-domain certificate witness.
code/affine_domain_proof.py	Restricted independent affine-domain proof generator/checker.
code/independent_checks.py	Exact mathematical oracles, separate from Wolfram execution.
tests/test_*.py	Source-transformation and proof-checker tests.
evidence/	Native observations, comparison outputs, execution scope, independent results, and logs.

The archive does not contain a complete repository checkout, a Wolfram engine, or a rebuilt packet. It contains the review and the code needed to reproduce and assess its proposed changes.

9.2 Preview and apply the canonical changes

With a local clean Git checkout at the stated pin:

```
python code/apply_fixes.py /path/to/Asymptotic
python code/apply_fixes.py /path/to/Asymptotic --apply
```

The default is a unified-diff preview. The apply mode verifies the exact HEAD, refuses staged or working-tree changes to either target source, requires each anchor once, creates backups, and updates only the canonical modules. It does not edit the generated standalone directly. From the repository root, follow with:

```
python validation/build_standalone.py
```

Then run the native package suites and compare the regenerated distribution with the reviewed transformation. These commands describe the intended local workflow; the upstream builder was not executed during this review.

9.3 Run the native focused checks

In a fresh Wolfram process, supply the pinned standalone file:

```
wolframscript -file code/native_probe.wls /path/to/AsymptoticInverse.wl
wolframscript -file code/native_probe.wls /path/to/AsymptoticInverse.wl --patched
```

The first command tests the unmodified input against desired contracts and is expected to expose the two baseline failures. The second changes only temporary text before loading. The runner records the caller-supplied input hash and loaded working-copy hash; it does not infer that an arbitrary local file belongs to the pin merely because its filename matches. Use the provided repository provenance to establish that separately.

A failure exit code from the baseline is intentional evidence, not a failed installation. Service/-transport failures are different again and must not be converted into numerical or semantic conclusions. The connected native execution in this report evaluated the same checks directly; the local command-line runner's file-I/O shell was not itself executed by a local Wolfram installation.

9.4 Run independent checks and rebuild the article

The Python mathematical oracles require SymPy; the proof checker and transformation tests otherwise use the standard library. From the archive root:

```
python code/independent_checks.py
python -m unittest discover -s tests -p "test_*.py" -v
cd article
pdflatex -interaction=nonstopmode -halt-on-error asymptotic_delta_audit.tex
pdflatex -interaction=nonstopmode -halt-on-error asymptotic_delta_audit.tex
```

The source uses standard TeX packages and no external figures or bibliography database. The second LaTeX run resolves the contents and cross-references. The included PDF was rendered and visually inspected after building.

10 Conclusion

The repository’s specialized inverse algorithms and retained mathematical metadata remain useful complements to Wolfram Language. The native comparisons show both substantial overlap and a concrete successful specialization; neither justifies a blanket superiority claim.

The most consequential new result is that correct rational root arithmetic can coexist with an incorrect domain certificate. The observed failure is not cured by more precision and is not merely the old constructor-provenance example: certificate-time symbolic refinement assumes the very condition it is meant to prove. A one-line option change closes that path, and a restricted, replayable domain witness provides a concrete next design step.

The sparse flat-sector finding is smaller but similarly focused. The multiplication loop already knows which pairs are exactly zero; its budget gate should use the same knowledge. The supplied change preserves the essential distinction between exact zero and unknown remainder, removes the observed unnecessary refusal, and leaves the coefficient and tail algebra untouched.

Both fixes have native positive and negative controls. The remaining release work is clearly bounded: canonical regeneration and broader neighboring-path regression, not a claim that a ten-check audit establishes correctness of the whole package. That is the appropriate balance between actionable repairs and an honest assessment of their evidence.

A Source coverage and prior-review map

Source area	Inspection depth and role
Kernel/AsymptoticInverse.wl and generated standalone	Core representation, dispatch, key precision and inversion paths; broad navigation across module boundaries. Not a formal proof of every branch.
Kernel/InverseCertificates.wl	Detailed proof arithmetic, condition handling, source interval, and refinement control; N01 traced and reproduced.
Kernel/FlatSectorOperations.wl	Detailed representation, zero test, multiplication, budgets, uncertainty, and selected derivative paths; N02 traced and reproduced.
Kernel/SeriesOperations.wl	Shared representation, arithmetic, composition, differentiation, and refinement paths.
Kernel/NativeSpecialFunctions.wl	Structured native import, carriers, error transport, domain admission, and exactness checks.
Kernel/SpecialFunctionIdentities.wl	Exact normalizations, finite hypergeometric/Bessel identities, and side-condition handling.
Kernel/DirichletSpecialFunctions.wl	Direct zeta/Lerch mechanisms and tail bounds.
Kernel/FourierCoefficients.wl	Coefficient/frequency algebra, reality, budgets, and principal construction paths.

Source area	Inspection depth and role
Kernel/GammaInverseOperations.wl	Selected inverse-core representation, residual, operation, and numerical paths.
Kernel/ExponentialCorePerturbation.wl	Growing-core recognition, exact inverse selection, coefficients, error transport, and native positive control.
Kernel/NumericalInverseChecks.wl	Source-domain checking, observable reconstruction, precision, and evidence labeling.
Other operations and coordinate families	Selected standalone cross-references and dispatch reads; not exhaustive independent validation.
Review index, status, and R1–R9	Full index/status crosswalk; executive inventories and targeted relevant arguments. No claim to have executed archived review scripts.

R1–R6, R8 and R9 principally reviewed the earlier 07a9781 baseline. R7 identifies 75de875 as its baseline. The present review uses 921387e, after current status entries and some targeted fixes were recorded. The distinction is important for C01/C02 and for source hashes. Each reference below points to the version of the relevant file in the *present pinned repository*, while the old article itself retains its original stated audit baseline.[3, 16, 17, 18, 19, 20, 21, 22, 23, 24]

B Interpreting negative results and nonclaims

The unevaluated native irrational inverse request is a narrow runtime observation. The failed endpoint fixture was an invalid-center test, not a certificate-domain bug. The initial 45-budget flat experiment hit a legitimate input-representation budget, not N02. The exponential-core metadata suspicion was rejected by source/runtime evidence. The service failures are transport failures, not package failures.

These distinctions are retained because an audit can accumulate plausible-looking “findings” faster than it accumulates evidence. The two numbered findings are the ones that survived source tracing, native reproduction, mathematical checking, comparison with prior work, and targeted repair. The broader implementation discussion is context and comparison, not an inflated list of rediscovered defects.

References

- [1] Vladimir Reshetnikov, *AsymptoticInverse README*, pinned revision 921387e. README.md.
- [2] *Code review reports index*. code-review/README.md.
- [3] *Consolidated code-review status and finding registry*. docs/development/CODE_REVIEW_STATUS.md.
- [4] *Canonical core and generated distribution*. AsymptoticInverse/Kernel/AsymptoticInverse.wl; AsymptoticInverse.wl.
- [5] *Shared series operations*. AsymptoticInverse/Kernel/SeriesOperations.wl.
- [6] *Incremental, grouped, and Newton inversion*. AsymptoticInverse/Kernel/IncrementalInverse.wl.
- [7] *Exact inverse certificates*. AsymptoticInverse/Kernel/InverseCertificates.wl.
- [8] *Flat-sector operations*. AsymptoticInverse/Kernel/FlatSectorOperations.wl.
- [9] *Native special-function adapter*. AsymptoticInverse/Kernel/NativeSpecialFunctions.wl.
- [10] *Exact special-function normalizations*. AsymptoticInverse/Kernel/SpecialFunctionIdentities.wl.
- [11] *Dirichlet special-function constructions*. AsymptoticInverse/Kernel/DirichletSpecialFunctions.wl.
- [12] *Fourier coefficient calculus*. AsymptoticInverse/Kernel/FourierCoefficients.wl.
- [13] *Gamma inverse operations and retained-core construction*. AsymptoticInverse/Kernel/GammaInverseOperations.wl; AsymptoticInverse.wl.
- [14] *Exponential-core perturbations*. AsymptoticInverse/Kernel/ExponentialCorePerturbation.wl.
- [15] *Numerical inverse comparisons*. AsymptoticInverse/Kernel/NumericalInverseChecks.wl.
- [16] *Code review 1: Mathematical and engineering audit*. code-review/code-review-1/article/article.tex.
- [17] *Code review 2: Technical review and Wolfram Language comparison*. code-review/code-review-2/article/asymptotic-review.tex.
- [18] *Code review 3: Source audit and Wolfram Language comparison*. code-review/code-review-3/article/asymptotic-audit.tex. In particular, its ambient-assumption finding F04.
- [19] *Code review 4: Source-pinned technical audit*. code-review/code-review-4/article.tex.
- [20] *Code review 5: Engineering and mathematical audit*. code-review/code-review-5/article/asymptotic-audit.tex.
- [21] *Code review 6: Correctness, contracts, and engineering audit*. code-review/code-review-6/article/article.tex.
- [22] *Code review 7: Technical audit and development roadmap*. code-review/code-review-7/article/asymptotic-audit.tex; code-review/code-review-7/article/03-findings.tex.
- [23] *Code review 8: Repository audit*. code-review/code-review-8/article/asymptotic_repository_audit.tex. In particular, its ambient-assumption finding F01.
- [24] *Code review 9: Source audit and development roadmap*. code-review/code-review-9/article.tex.
- [25] Wolfram Research, *Series*. Official Wolfram Language documentation, accessed 9 September 2026. <https://reference.wolfram.com/language/ref/Series.html>.
- [26] Wolfram Research, *SeriesData*. Official documentation, especially rational powers, logarithmic coefficients, and outside carriers. <https://reference.wolfram.com/language/ref/SeriesData.html>.

- [27] Wolfram Research, *InverseSeries*. Official documentation. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [28] Wolfram Research, *ComposeSeries*. Official documentation. <https://reference.wolfram.com/language/ref/ComposeSeries.html>.
- [29] Wolfram Research, *Asymptotic*. Official documentation, including inferred scales and broader analysis inputs. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [30] Wolfram Research, *AsymptoticSolve*. Official documentation, including real-valued solutions and equation systems. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [31] Wolfram Research, *ProductLog*. Official documentation. <https://reference.wolfram.com/language/ref/ProductLog.html>.
- [32] Wolfram Research, *Refine*, Assumptions option examples: positional assumptions combine with `$Assumptions`; an explicit option prevents that default import. <https://reference.wolfram.com/language/ref/Refine.html>.